>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 7

Loops: while and do-while

Pre-test and post-test loops · sentinels · input validation · menus · number algorithms

```
> dotnet run
Your guess: 50
Higher!
Your guess: 75
Lower!
...
Your guess: 67
Correct! 7 attempts.
```

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the end of this week you can …

Write while loops

Pre-test loops: may run zero times. Trace them with a table.

Write do-while loops

Post-test loops: the body always runs at least once.

Choose the right loop

Counter- vs condition-controlled; convert for ⇄ while.

Validate input

Repeat until TryParse succeeds and the value is in range; menus.

Process numbers

% 10 and / 10 for digits, Euclid GCD/LCM, binary, Collatz.

Control the loop

while (true) + break, continue, infinite loops, loop invariants.

for loops count — but what if we cannot count?

Counter-controlled loop

```
// Week 6: the number of passes is KNOWN
int sum = 0;
for (int i = 1; i <= 10; i++)
{
    sum += i;
}
Console.WriteLine(sum);   // 55
```

for = counter-controlled. Start, end and step are known before the first pass: exactly 10 passes here.

How many passes will these need?

Class average
The teacher types scores until she types −1.

Guess the number
The game goes on until the guess is correct.

Savings account
How many years until 10 000 TL doubles?

Unknown in advance → `while` / `do-while`



PART 1

while and do-while

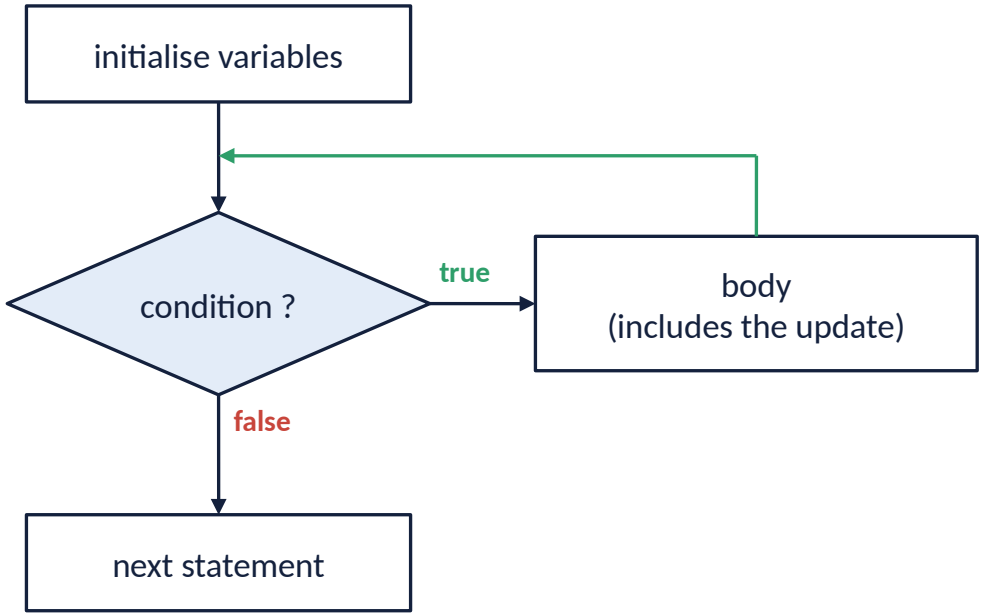
Pre-test and post-test loops · flowcharts · trace tables · for ⇄ while

while: test first, then run the body

Syntax

```
while (condition)
{
    // body: runs again and again
    // as long as condition is true
}
// next statement after the loop
```

1. Test the condition (a bool expression).
2. true → run the body once, then go back to 1.
3. false → continue after the loop.



Pre-test loop: 0, 1, 2, … passes. The body may never run.

Trace table: one row per test of the condition

How long until 100 TL passes 1000 TL?

```
int money = 100;
int years = 0;
while (money < 1000)
{
    money *= 2;
    years++;
}
Console.WriteLine($"{years} years, {money} TL");
```

Console

```
4 years, 1600 TL
```

Test	money	years	money < 1000	Action
1	100	0	true	run body
2	200	1	true	run body
3	400	2	true	run body
4	800	3	true	run body
5	1600	4	false	exit loop

5 tests, 4 passes. A while loop always tests its condition once more than it runs the body. The last test is the false one.

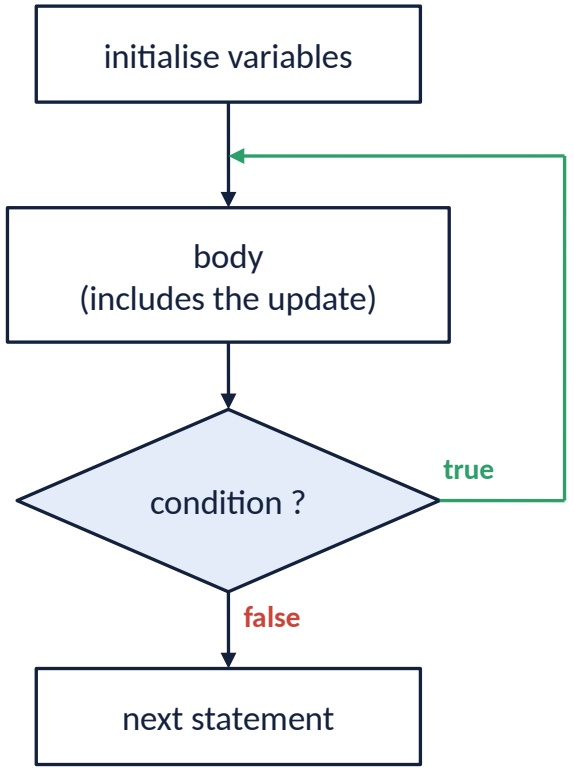
do-while: run the body first, then test

Syntax

```
do
{
    // body: runs FIRST,
    // then the condition is tested
}
while (condition);   // semicolon!
// next statement after the loop
```

At least one pass
The body runs before the first test.

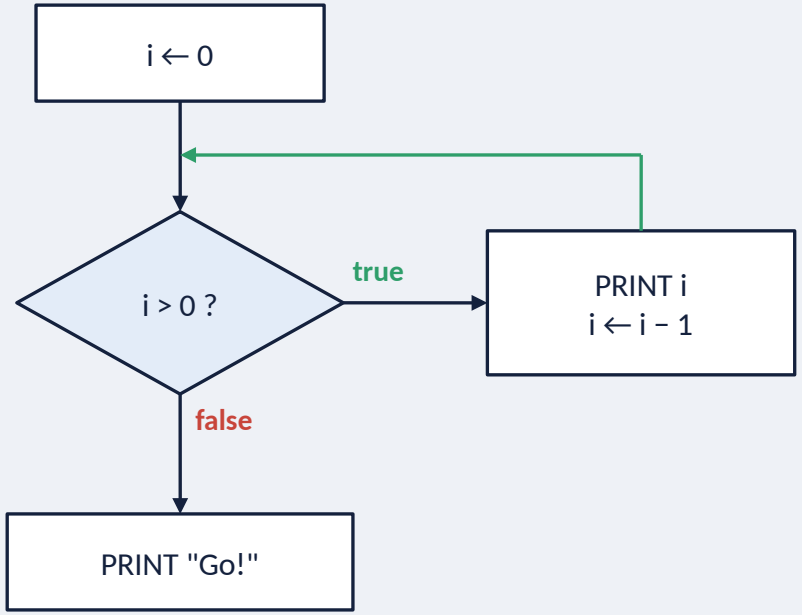
while (...); ends with a semicolon
Only in do-while! Missing → CS1002.



Post-test
1, 2, 3, … passes.
Never zero.

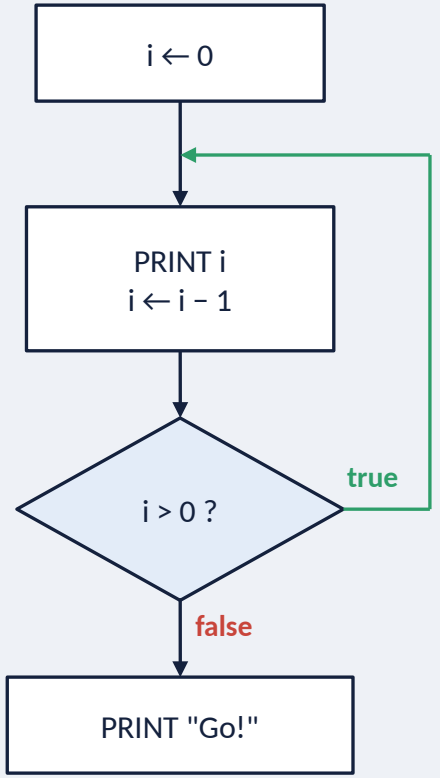
Same condition, same start value: i = 0

while: diamond first



0 passes → output: Go !

do-while: body first



1 pass → output:
0 Go !

One countdown, three loops

Program.cs (excerpt)

```
Console.Write("while    : ");
int i = n;
while (i > 0)
{
    Console.Write($"{i} ");
    i--;
}
Console.WriteLine("Go!");

Console.Write("do-while : ");
int j = n;
do
{
    Console.Write($"{j} ");
    j--;
}
while (j > 0);
Console.WriteLine("Go!");
```

Console

```
Start the countdown from: 3
while    : 3 2 1 Go!
do-while : 3 2 1 Go!
for      : 3 2 1 Go!
```

Console

```
Start the countdown from: 0
while    : Go!
do-while : 0 Go!
for      : Go!
```



n = 0: only do-while prints 0.

Every for loop is a while loop in disguise

```for
int total = 0;
for (int k = 1; k <= 5; k++)
{
    total += k;
}
```



```while
int total = 0;
int k = 1;          // 1 init
while (k <= 5)      // 2 condition
{
    total += k;
    k++;            // 3 update
}
```

Part	for	while
1 init	inside the header	before the loop
2 condition	in the header	in while (…)
3 update	in the header	last line of the body
k after the loop	does not exist	still exists: k = 6



Both give total = 15. In while, a continue placed before k++ skips the update → infinite loop.

>_ WHICH LOOP WHEN?

Choose the loop that states your intent

Situation	Example	Best choice
Number of passes known before the loop	print 1..N, read exactly N scores	`for`
Repeat until a special stop value	read scores until −1 (sentinel)	`while`
Zero passes must be possible	process digits while n > 0	`while`
Body must run at least once	menu, "ask again" input validation	`do-while`
Stop decision is in the middle of the body	guess game: read, check, break	`while (true) + break`



Counter-controlled

We know HOW MANY passes: a counter goes from start to end.



Condition-controlled

We know WHEN to stop, not how many passes: sentinel, flag, result.



PART 2

Sentinels, validation and menus

Loops that talk to the user: read until −1, ask again until valid, show a menu until Exit

Read until a special value: the sentinel

Pseudocode

```
START
  count ← 0
  sum ← 0
  READ score            (priming read)
  WHILE score ≠ −1
    count ← count + 1
    sum ← sum + score
    READ score          (next value)
  END WHILE
  IF count > 0 THEN
    PRINT sum / count
  ELSE
    PRINT "No scores"
  END IF
END
```

READ #	score	score ≠ −1	count	sum
1 (priming)	72	true	1	72
2	85	true	2	157
3	90	true	3	247
4	64	true	4	311
5	−1	false	4	311

`average = 311 / 4 = 77.75`



The sentinel is never counted: it is tested before it is used.

Class average with the sentinel –1

Program.cs (excerpt)

```
const int Sentinel = -1;
int count = 0;
int sum = 0;
int highest = 0;

Console.WriteLine($"Enter scores (0-100), {Sentinel} to finish.");
Console.Write("Score: ");
int score = int.Parse(Console.ReadLine());  // priming read

while (score != Sentinel)
{
    count++;
    sum += score;
    if (count == 1 || score > highest)
    {
        highest = score;        // first score or a new record
    }
    Console.Write("Score: ");
    score = int.Parse(Console.ReadLine());  // the NEXT value
}
```

Console

```
Enter scores (0-100), -1 to finish.
Score: 72
Score: 85
Score: 90
Score: 64
Score: -1
Count: 4  Sum: 311
Average: 77.75  Highest: 90
```

Console

```
Enter scores (0-100), -1 to finish.
Score: -1
No scores entered.
```



No if (count == 0)? 0.0 / 0 = NaN

Ask again until the input is valid

Program.cs (part a)

```
int score;
bool ok;
do
{
    Console.Write("Exam score (0-100): ");
    ok = int.TryParse(Console.ReadLine(), out score);
    if (!ok || score < 0 || score > 100)
    {
        Console.WriteLine("  Invalid: whole number 0-100 only.");
    }
}
while (!ok || score < 0 || score > 100);
```

Console

```
Exam score (0-100): abc
  Invalid: whole number 0-100 only.
Exam score (0-100): 120
  Invalid: whole number 0-100 only.
Exam score (0-100): 85
Height in metres (0.5-2.5): 1,75
  Invalid: example 1.75
Height in metres (0.5-2.5): 0
  Invalid: example 1.75
Height in metres (0.5-2.5): 1.75
Accepted: 85 points, 1.75 m
```



Repeat while NOT valid

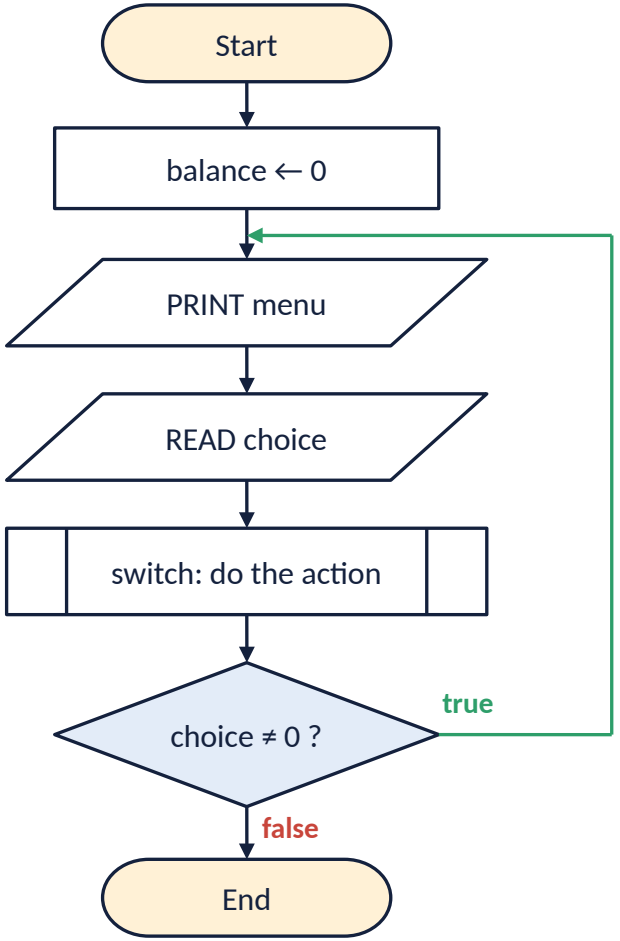
!ok || score < 0 || score > 100



If TryParse fails, the out value is 0

so always test ok before the range.

A menu is a do-while around a switch



do-while
The menu must be shown at least once.

switch (Week 5)
One case per menu option; default catches everything else.

Exit option
The loop condition is choice != 0.

TryParse fails: choice = 0 = Exit!
Set choice = -1 instead, so default reports the error.

Okan Mini Bank: menu loop in C#

Program.cs (excerpt)

```
do
{
    Console.WriteLine("1) Balance 2) Deposit 3) Withdraw 0) Exit");
    Console.Write("Choice: ");
    if (!int.TryParse(Console.ReadLine(), out choice))
    {
        choice = -1;        // not a number -> default case
    }
    switch (choice)
    {
        // case 1, 2, 3: balance, deposit, withdraw (see Program.cs)
        case 0:
            Console.WriteLine("Goodbye!");
            break;
        default:
            Console.WriteLine("Unknown option, try again.");
            break;
    }
}
while (choice != 0);
```

Console

```
=== OKAN MINI BANK ===
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 2
Amount to deposit: 500
New balance: 500.00 TL
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 3
Amount to withdraw: 750
Insufficient funds.
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: x
Unknown option, try again.
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 0
Goodbye!
```



"x" is not 0: choice = -1 → default.



PART 3

Number algorithms with while

Euclid GCD & LCM · digits with % 10 and / 10 · decimal → binary · Collatz

Euclid's GCD (Week 1) in C# — and the LCM

Program.cs (excerpt, inside else)

```
int x = a;          // copies: a and b are needed for the LCM
int y = b;
Console.WriteLine($"{"x",6} {"y",6} {"x % y",7}");
while (y != 0)
{
    int r = x % y;
    Console.WriteLine($"{x,6} {y,6} {r,7}");
    x = y;
    y = r;
}
int gcd = x;
long lcm = (long)a / gcd * b;   // divide first, then multiply
Console.WriteLine($"GCD({a}, {b}) = {gcd}");
Console.WriteLine($"LCM({a}, {b}) = {lcm}");
```

Console

```
First positive integer a : 48
Second positive integer b: 18
     x      y   x % y
    48     18      12
    18     12       6
    12      6       0
GCD(48, 18) = 6
LCM(48, 18) = 144
```

gcd(48,18) = gcd(18,12)
= gcd(12,6) = gcd(6,0) = 6

lcm = a / gcd · b
48 / 6 · 18 = 144

>_ PEELING OFF DIGITS

% 10 gives the last digit, / 10 removes it

4	0	7	1

```
4071 % 10
```

→ 1 (last digit)

```
4071 / 10
```

→ 407 (drop it)

rest	digit	count	sum	reversed
4071	1	1	1	1
407	7	2	8	17
40	0	3	8	170
4	4	4	12	1704
0	stop: rest > 0 is false			

Values at the END of each pass

`reversed = reversed * 10 + digit`
appends the new digit on the right: 17 → 170 → 1704.

4 digits, sum 12, reverse 1704
1704 ≠ 4071 → not a palindrome

Digit count, digit sum, reverse, palindrome

Program.cs (excerpt)

```
int rest = n;           // working copy; n is kept for the test
int digitCount = 0;
int digitSum = 0;
long reversed = 0;      // long: the reverse may not fit in an int

// do-while, so that n = 0 is counted as a 1-digit number
do
{
    int digit = rest % 10;              // last digit
    digitCount++;
    digitSum += digit;
    reversed = reversed * 10 + digit;   // append on the right
    rest /= 10;                         // drop the last digit
}
while (rest > 0);
```

Console

```
Enter a non-negative integer: -5
Enter a non-negative integer: 12321
Digits : 5
Sum    : 9
Reverse: 12321
12321 is a palindrome.
```

Console

```
Enter a non-negative integer: 0
Digits : 1
Sum    : 0
Reverse: 0
0 is a palindrome.
```

do-while: 0 has 1 digit.

Decimal to binary: divide by 2 until 0

Program.cs (excerpt)

```
int rest = n;
string binary = "";
do
{
    int bit = rest % 2;
    Console.WriteLine($"{rest} / 2 = {rest / 2}, remainder {bit}");
    binary = bit + binary;      // new bit goes to the LEFT
    rest /= 2;
}
while (rest > 0);

Console.WriteLine($"{n} in binary = {binary}");
Console.WriteLine($"Built-in check: {Convert.ToString(n, 2)}");
```

2026 → 11111101010 (1024+512+256+128+64+32+8+2)

Console

```
Non-negative integer: 13
13 / 2 = 6, remainder 1
6 / 2 = 3, remainder 0
3 / 2 = 1, remainder 1
1 / 2 = 0, remainder 1
13 in binary = 1101
Built-in check: 1101
```

rest	rest % 2	bit string
13	1	1
6	0	01
3	1	101
1	1	1101

Collatz: nobody knows how many passes

Program.cs (excerpt)

```
while (n != 1)
{
    if (n % 2 == 0)
    {
        n = n / 2;
    }
    else
    {
        n = 3 * n + 1;
    }
    steps++;
    if (n > peak)
    {
        peak = n;
    }
    // 10 numbers per line keeps long sequences readable
    Console.Write(steps % 10 == 0 ? $"\n{n}" : $" {n}");
}
```

Console

```
Starting value (>= 1): 6
6 3 10 5 16 8 4 2 1
8 steps, highest value 16
```

Console

```
Starting value (>= 1): 0
Starting value (>= 1): 27
27 82 41 124 62 31 94 47 142 71
214 107 322 161 484 242 121 364 182 91
274 137 412 206 103 310 155 466 233 700
350 175 526 263 790 395 1186 593 1780 890
445 1336 668 334 167 502 251 754 377 1132
566 283 850 425 1276 638 319 958 479 1438
719 2158 1079 3238 1619 4858 2429 7288 3644 1822
911 2734 1367 4102 2051 6154 3077 9232 4616 2308
1154 577 1732 866 433 1300 650 325 976 488
244 122 61 184 92 46 23 70 35 106
53 160 80 40 20 10 5 16 8 4
2 1
111 steps, highest value 9232
```



PART 4

Games, security and pitfalls

Random · while (true) + break · continue · attempts · infinite loops · loop invariants

Guess the number: while (true) + break

Program.cs (excerpt)

```
while (true)
{
    Console.Write("Your guess: ");
    if (!int.TryParse(Console.ReadLine(), out int guess))
    {
        Console.WriteLine("Please type a whole number.");
        continue;           // back to the top; attempt not counted
    }
    attempts++;
    if (guess < secret)
    {
        Console.WriteLine("Higher!");
    }
    // else if (guess > secret): "Lower!" (same shape)
    else
    {
        Console.WriteLine($"Correct! {attempts} attempts.");
        break;              // leave the loop immediately
    }
}
```

secret: 1 … 100

```
Random.Shared.Next(1, 101)
```

continue
back to the top, skip the rest

break
leave the loop immediately

attempts++
counts only valid guesses

Reproducible runs with a fixed seed

Console (test run, seed 42)

```
I am thinking of a number between 1 and 100.
Your guess: 50
Higher!
Your guess: 75
Lower!
Your guess: abc
Please type a whole number.
Your guess: 62
Higher!
Your guess: 68
Lower!
Your guess: 65
Higher!
Your guess: 66
Higher!
Your guess: 67
Correct! 7 attempts.
```

Test line (swap for the first line)

```
// int secret = Random.Shared.Next(1, 101);
int secret = new Random(42).Next(1, 101);
```

Same seed, same secret

Seed 42 gave secret 67 in every run.

Halving strategy (Week 1)

Always guess the middle of what is left.

Worst case for 1..100: $\lceil \log_2 101 \rceil = 7$ guesses

Now the computer guesses: binary search

Program.cs (excerpt)

```
do
{
    guess = (low + high) / 2;       // middle of the range
    guesses++;
    Console.Write($"#{guesses}: {low}..{high} -> {guess}  ");
    if (guess < secret)
    {
        Console.WriteLine("higher");
        low = guess + 1;
    }
    // else if (guess > secret): "lower", high = guess - 1
    // else: "correct!"
}
while (guess != secret);
```

Console

```
Secret number for the computer (1-100): 67
#1: 1..100 -> 50  higher
#2: 51..100 -> 75  lower
#3: 51..74 -> 62  higher
#4: 63..74 -> 68  lower
#5: 63..67 -> 65  higher
#6: 66..67 -> 66  higher
#7: 67..67 -> 67  correct!
Found 67 in 7 guesses.
```

All 100 secrets tested: guesses needed

guesses	1	2	3	4	5	6	7
secrets	1	2	4	8	16	32	37

maximum 7, average 5.8

Password with three attempts

Program.cs (excerpt)

```
const string Password = "okan2026";
const int MaxAttempts = 3;
int attempts = 0;
bool granted = false;
while (attempts < MaxAttempts && !granted)
{
    Console.Write("Password: ");
    string entry = Console.ReadLine();
    attempts++;
    if (entry == Password)
    {
        granted = true;
    }
    else if (attempts < MaxAttempts)
    {
        int left = MaxAttempts - attempts;
        Console.WriteLine($"Wrong! {left} attempt(s) left.");
    }
}
```

Console

```
Password: okan2025
Wrong! 2 attempt(s) left.
Password: okan2026
Access granted (attempt 2).
```

Console

```
Password: 1234
Wrong! 2 attempt(s) left.
Password: password
Wrong! 1 attempt(s) left.
Password: Okan2026
Wrong password. Account locked.
```



Two exits: granted is true OR attempts reached 3.

How many years until the money doubles?

Program.cs (excerpt)

```
decimal rate;
do
{
    Console.Write("Annual interest rate in % (> 0): ");
    rate = decimal.Parse(Console.ReadLine());
}
while (rate <= 0);      // 0 % -> the loop below never ends!

decimal balance = principal;
int years = 0;
Console.WriteLine($"{"Year",4} {"Balance (TL)",14}");
while (balance < 2 * principal)
{
    balance += balance * rate / 100;   // one year of interest
    years++;
    Console.WriteLine($"{years,4} {balance,14:F2}");
}
Console.WriteLine($"The deposit doubles after {years} years.");
Console.WriteLine($"Rule of 72: about {72 / rate:F1} years");
```

Console

```
Deposit (TL): 10000
Annual interest rate in % (> 0): 12
Year   Balance (TL)
   1       11200.00
   2       12544.00
   3       14049.28
   4       15735.19
   5       17623.42
   6       19738.23
   7       22106.81
The deposit doubles after 7 years.
Rule of 72: about 6.0 years
```



Rule of 72: years ≈ 72 / r. At 12 %: 6.0 estimated, 7 by the loop.

When the condition never becomes false

① No update → prints 1 forever

```
int i = 1;
while (i <= 10)
{
    Console.WriteLine(i);
}
```

② Stray ; → hangs silently (CS0642)

```
int i = 1;
while (i <= 10);
{
    Console.WriteLine(i);
    i++;
}
```

③ Condition never false

```
int k = 1;
while (k != 10)   // 1, 3, 5, …
{
    k += 2;
}
```

④ Nothing changes: rate 0 %

```
while (balance < 2 * principal)
{
    balance += balance * 0 / 100;
}
```

Stuck? Press Ctrl+C in the terminal to stop the running program.

What stays true on every pass?

Sum 1..n

```
sum ← 0
i ← 1
WHILE i ≤ n
  sum ← sum + i
  i ← i + 1
END WHILE
```

Invariant: sum = 1 + ... + (i − 1)
At the end i = n + 1, so sum = 1 + … + n.

Euclid

```
WHILE y ≠ 0
  r ← x mod y
  x ← y
  y ← r
END WHILE
```

Invariant: gcd(x, y) = gcd(a, b)
At the end y = 0, and gcd(x, 0) = x.

Halving

```
WHILE guess ≠ secret
  guess ← (low+high) div 2
  … low ← guess + 1
  … high ← guess − 1
END WHILE
```

Invariant: low ≤ secret ≤ high
The range shrinks each pass, so the loop ends.



true before the loop + stays true after each pass + exit condition ⇒ correct result

Seven loop bugs and how to fix them

Mistake	Symptom	Fix
`while (i <= 10);`	hangs; warning CS0642	remove the ;
no update in the body	infinite loop	change a variable of the condition
`do { … } while (x > 0)`	error CS1002: ; expected	end do-while with ;
`while (x = 5)`	error CS0029 (int → bool)	compare with ==
READ at the top of a sentinel loop	sentinel added to the sum	priming read + READ at the end
range check without !ok	"abc" accepted as 0	test !ok || out of range
< instead of <= (or vice versa)	one pass too many / too few	trace table, test the boundaries

>_ PREDICT THE OUTPUT

Quiz: what is printed? How many passes?

Q1

```
int n = 10;
while (n > 1)
{
    n = n / 2;
    Console.Write($"{n} ");
}
```

Q2

```
int x = 100;
do
{
    Console.Write($"{x} ");
    x += 10;
}
while (x < 50);
Console.WriteLine($"| {x}");
```

Q3

```
int c = 0;
int m = 1;
while (m < 100)
{
    m *= 3;
    c++;
}
Console.WriteLine($"{m} {c}");
```

? **Write the output exactly, and count the passes of the body.**

Answers (checked by running the code)

Q1

```
5 2 1
```

n before	n after	printed
10	5	5
5	2	2
2	1	1

`1 > 1 is false: stop`

Q2

```
100 | 110
```

do-while runs once
prints 100, x becomes 110, then 110 < 50 is false → stop.

Q3

```
243 5
```

pass	m	c
1	3	1
2	9	2
3	27	3
4	81	4
5	243	5



Q1: 3 passes, 4 tests. Q2: 1 pass. Q3: 5 passes.

Your turn (pairs, 25 minutes)

E1 · Even and odd ★

Read integers until 0 (sentinel). Print how many were even and how many were odd.

E2 · Month days ★

Read a month 1–12 with do-while + TryParse (ask again if invalid). Print its number of days (Feb = 28) with switch.

E3 · Count a digit ★★

Read n and a digit d. Print how many times d occurs in n (e.g. 1 in 10101 → 3). Careful with n = 0.

E4 · from for to while ★★

Rewrite the Week 6 factorial as a while loop. Then find the smallest n with n! > 1 000 000.

>_ HOMEWORK

Before next week (midterm review!)

H1 · Unit converter menu

do-while + switch: 1) km → mile, 2) °C → °F, 3) kg → lb, 0) Exit. Validate every number with TryParse.

H2 · Prime check with while

Read n ≥ 2 and test divisors d while d · d ≤ n. Stop at the first divisor found.

H3 · Better guessing game

At most 7 attempts; after each game ask "Play again? (y/n)" with a do-while around the game.

H4 · Binary to decimal

Read a number made of 0s and 1s (e.g. 1101) as an int and convert it with % 10 and / 10 (1101 → 13).

Submit the four Program.cs files on the course group; include one console run for each.

>_ SUMMARY

Key Takeaways

01 while tests BEFORE the body (0 or more passes); do-while tests AFTER (1 or more passes).

02 Use for when you can count the passes; while/do-while when you only know when to stop.

03 Sentinel loops: priming read before the loop, read the next value at the end of the body.

04 Validation: repeat while !ok || out of range. Menus: do-while around a switch.

05 % 10 and / 10 peel digits; Euclid, binary and Collatz are while loops too.

06 Every loop needs progress toward its exit. Stuck? Ctrl+C, then trace the variables.

NEXT WEEK

Week 8: Midterm review & practice exam



MCMP113
Course materials